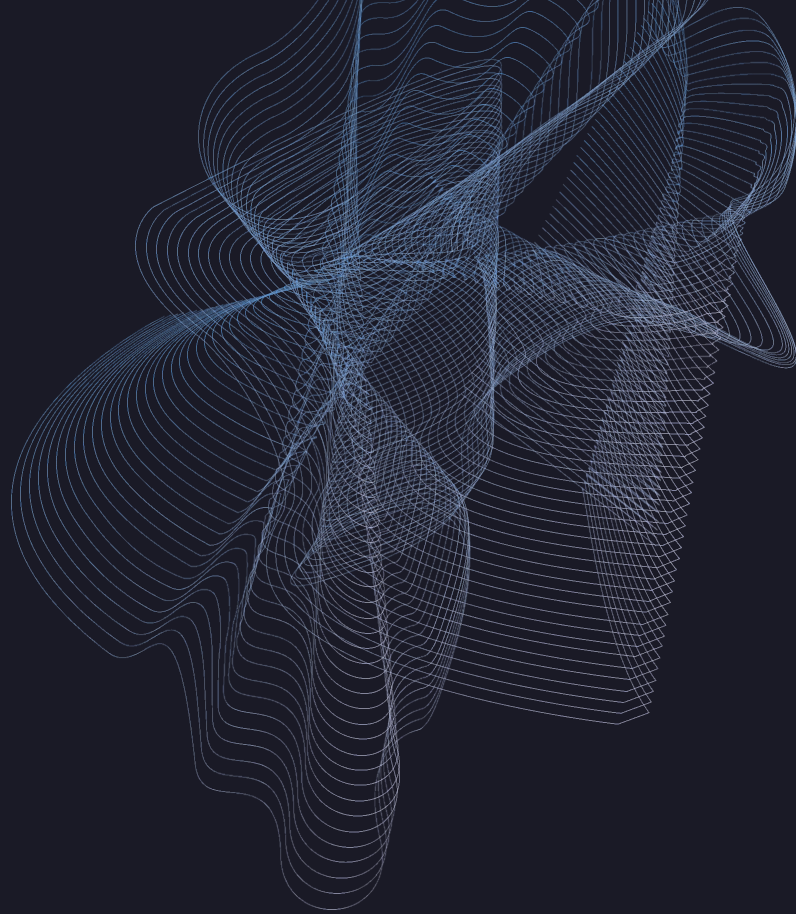


OFF-BY-ONE 2026

# Beyond XNU: Agentic Hunting for Vulnerabilities in TXM

stefan@calif.io · Off-by-One, Singapore

© 2026 calif.io · Off-by-One · Singapore

# whoami

## Stefan Esser · @i0n1c

25+ years in the field · since 2010 focused on Apple

Taught XNU kernel exploitation for years

Security Researcher at calif.io

Apple Silicon: XNU · SPTM/TXM · Secure Kernel



# Content

1
Why TXM

2
Rehosting TXM in Userland

3
The GLx Runner

4
Diffing TXM Across Releases

5
Auditing TXM with Agents

6
From Rehost to Fuzzers

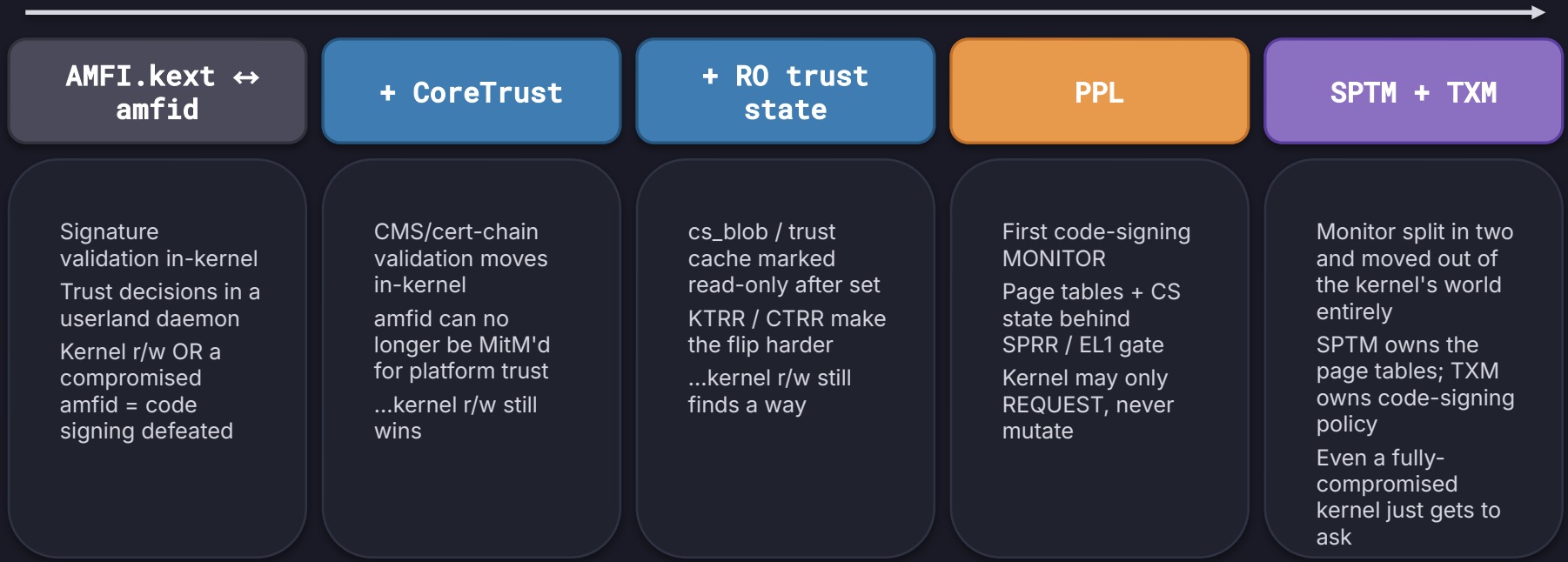
1

# Why TXM

© 2026 calif.io · Off-by-One · Singapore

# 15 years of moving the goalposts

each step made the kernel less able to lie about what code it runs

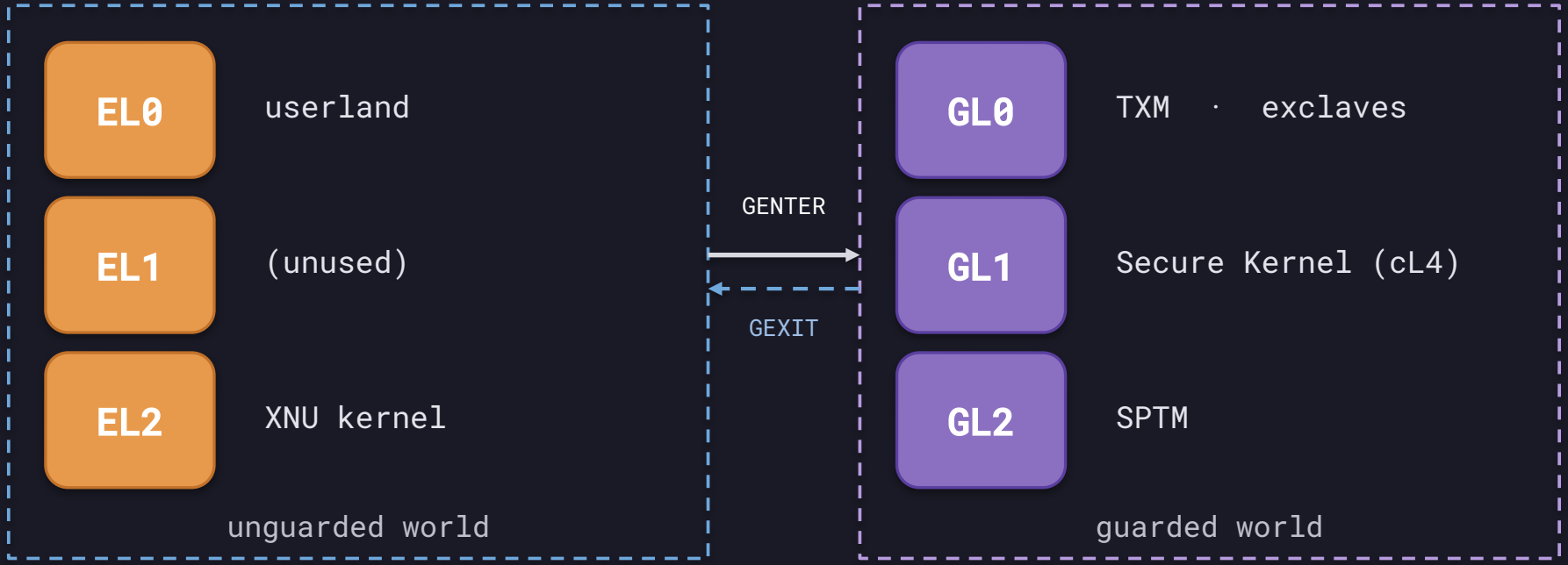



each generation: one more thing kernel r/w can no longer touch.

© 2026 calif.io · Off-by-One · Singapore

# Where SPTM + TXM actually live

Apple's guarded world: a lateral set of exception levels the kernel can't reach




GXF adds a lateral guarded world (GL0–GL2); you cross via GENTER / GEXIT. TXM (our target) runs at GL0.

© 2026 calif.io · Off-by-One · Singapore

# The Code-Signing Monitor

XNU abstracts the monitor behind CSM_PREFIX(...): bound to txm / ppl / xnu at build time

## What it is

Code-signing enforcement + trust state live in an isolated monitor, not in kernel-writable memory

The kernel may only REQUEST changes via monitor calls, never mutate the state directly

On SPTM platforms that monitor is TXM (PPL before it)

**xnu · bsd/sys/code_signing_internal.h**

```
// abstract the monitor; bind it at build time
#if   CONFIG_SPTM            // TrustedExecutionMonitor
#  define CODE_SIGNING_MONITOR_PREFIX  txm
#elif PMAP_CS_PPL_MONITOR    // Page Protection Layer
#  define CODE_SIGNING_MONITOR_PREFIX  ppl
#else                        // no monitor at all
#  define CODE_SIGNING_MONITOR_PREFIX  xnu
#endif

#define CSM_PREFIX(name) \
        _CSM_PREFIX(CODE_SIGNING_MONITOR_PREFIX, name)

kern_return_t CSM_PREFIX(register_code_signature)(
        vm_address_t sig, vm_size_t size, ...);
```

**Where it lands in a real exec**

```
execve → exec_mach_imgact → …check_signature → ubc_cs_blob_add → cs_blob_create_validated
  → csm_register_code_signature()   → CSM_PREFIX(register_code_signature)()
     = txm_register_code_signature()   → TXM_CALL(kTXMKernelSelectorRegisterCodeSignature, …)
```

# Breaking code signing = breaking TXM

kernel r/w is no longer the finish line: TXM is

## What kernel r/w can NOT do

Mark an unsigned page executable: SPTM refuses the retype

Forge a code-signature object: the object lives in TXM's slab, typed by SPTM

Edit the trust cache: read-only, TXM-owned

Grant an entitlement: the entitlements context is TXM-owned

It can only ASK.  TXM says no.

## What that means for us

TXM is the mandatory next hop after XNU

It runs at GL0: the guarded world, not the kernel's world

It ships as a stripped binary blob nobody can just run

...which is exactly why it's worth studying.

© 2026 calif.io · Off-by-One · Singapore

# The TXM Dispatch Enum

one selector enum ≈ the whole kernel→TXM attack surface · txm_data_types.h

```
enum {   // kTXMKernelSelector<Name>
  // 0 = ResumeThread  (in SPTM.h)
  // --- global data ---
  GetLogInfo = 1,
  GetCodeSigningInfo,
  GetTrustCacheInfo,
  GetBuildVariant,
  EnterLockdownMode,
  EnableRestrictedMode,
  GetSecureChannelAddr,
  UpdateDeviceState,
  CompleteSecurityBootMode,
  // --- free list ---
  AddFreeListPage,
  GetFreeListPage,
  // --- trust cache ---
  LoadTrustCache,
  UnloadTrustCache,
  QueryTrustCache,
  QueryTrustCacheForREM,
  CheckTrustCacheRuntimeForUUID,
  // --- provisioning profiles ---
  RegisterProvisioningProfile,
  TrustProvisioningProfile,
  UnregisterProvisioningProfile,
  AssociateProvisioningProfile,
  DisassociateProvisioningProfile,
  // --- code signatures & policy ---
  RegisterCodeSignature,
  UnregisterCodeSignature,
  ValidateCodeSignature,
  ReconstituteCodeSignature,
  SetLocalSigningPublicKey, GetLocalSigningPublicKey,
  AuthorizeLocalSigningCDHash,
  AuthorizeCompilationServiceCDHash,
  MatchCompilationServiceCDHash,
  DeveloperModeToggle, AcquireSigningIdentifier,
  AssociateKernelEntitlements, AccelerateEntitlements,
  // --- address space ---
  SetSharedRegionBaseAddress,
  RegisterAddressSpace, UnregisterAddressSpace,
  AssociateCodeSignature, GetEntitlementsContext,
  ResolveKernelEntitlementsAddressSpace,
  AllowJITRegion, AssociateJITRegion,
  AllowInvalidCode, AssociateDebugRegion,
  // --- image4 / nonce ---
  Image4Dispatch, Image4GetExports,
  Image4SetReleaseType, Image4SetNonce,
  Image4SetBootNonceShadow,
  Image4RollNonce, Image4GetNonce,
  // --- special ---
  GL0ExceptionTest,
  kTXMKernelSelectorTotal
};
```

© 2026 calif.io · Off-by-One · Singapore

# The TXM binary

one file in the IPSW · one Mach-O · one symbol

```
IPSW
 └── Firmware/txm.macosx.release.im4p      ~232 KB
     Firmware/txm.iphoneos.release.im4p
      IMG4 payload  ·  cc 'trxm'  ·  LZFSE (bvx2)
          |  ipsw img4 extract
          ▼
     txm.bin    Mach-O arm64e  MH_EXECUTE  PIE
                576 KB macOS  ·  544 KB iOS
```

**6 segments (27.0 RC)**

| | | |
|---|---|---|
| __TEXT | 112 KB | const/strings |
| __DATA_CONST | 80 KB | tables |
| __TEXT_EXEC | 336 KB | the code |
| __TEXT_BOOT_EXEC | 32 KB | entry/vectors |
| __DATA | 16 KB | |
| __LINKEDIT | 32 B | |

## What you get for free

~2 350 strings: entitlement names, ~60 panic/overflow messages, 38 com.apple.* IDs, 7 libc source paths

~1 450 functions (IDA auto-analysis): none of them named

© 2026 calif.io · Off-by-One · Singapore

# …but Apple ships the headers

the binary has 1 symbol. The KDK has 2 570 lines of TXM private headers.

**KDK · PrivateHeaders/platform/TrustedExecutionMonitor/**

| | | |
|---|---|---|
| API.h | 613 | every selector |
| Return.h | 705 | every panic / error code |
| Types.h | 465 | slab · profile · trustcache · … |
| Global.h | 268 | every global → |
| Stacks.h | 129 | thread stack |
| SPTM.h | 188 | frame types |
| TryLock.h | 202 | |

\+ CodeSignature/ · CoreEntitlements/V2/ · sptm/sptm_common.h

**Global.h · TXMReadOnlyData_t (excerpt)**

```
typedef struct _TXMReadOnlyData {
    bool  readOnlyLockdown;
    bool  DITSupported, MTESupported;
    struct { bool virtualized; TXMDeviceFusing_t fusing;
             TXMDeviceType_t platform; } deviceType;
    TXMMutableVirtualRegion_t secureChannel;
    TrustCacheRuntime_t trustCacheReadOnlyRT;
    TrustCache_t  staticTrustCache0, staticTrustCache1;
    CSConfig_t    CSConfiguration;
    struct {
        bool allowModifiedCode;
        bool allowUnrestrictedDebugging;
        bool developerModeEnabled;
        bool relaxLockdownModeJIT;  …
    } exemptions;
    bool codeSigningDisabled;
} TXMReadOnlyData_t;
```

one struct applied in IDA and every field of gRO in __DATA_CONST has a name.

# ...and you can't just run it

GL0 only exists behind SPTM behind iBoot: there is no lldb here

## Why static-only RE is painful

Runs at GL0: a mode your CPU only enters via GENTER from GL2 (SPTM), which only iBoot loads

Talks to SPTM via SVC #0: in userland that's a Darwin syscall, on a bare VM it's an undefined trap

Talks to XNU via a dispatcher registered at boot: no XNU, no calls

No debugger attaches to GL0. No lldb, no dtrace, no printf you can see.

## So we need to

Make it runnable

Make it instrumentable

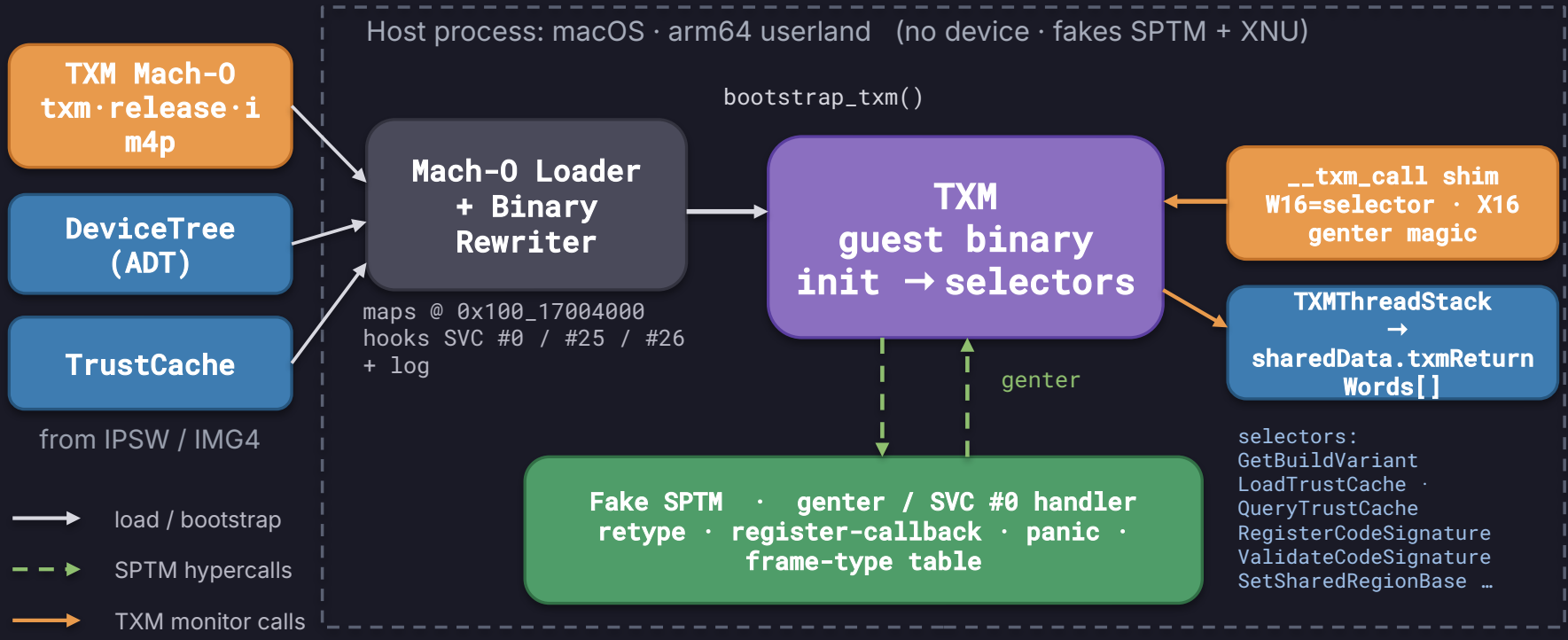
...on a normal Mac, without a device

2

# Rehosting TXM in Userland

# Rehosting TXM in Userland

the original, hand-built design: 2025, no coding agent



# GL0 → EL0: same CPU, wrong world

the ONE address change: top bits masked, everything below identical

```
// real TXM (GL0)              userland rehost (EL0)
__TEXT vmaddr:
  0xFFFFFFF017004000     →     0x0000010017004000

seg->vmaddr = seg->vmaddr - 0xFFFFFFF017004000
                          + 0x0000010017004000;
// live = ida_va - 0xFFFFFFF017004000 + 0x10017004000
```

## Why it just works

- Same low 40 bits → every ADRP / B / LDR-literal still resolves
- MAP_FIXED, no ASLR in the TXM window → stable across runs
- Only segment vmaddrs rewritten: no code relocation

## The one gotcha

0x10017004000 is ABOVE SPTM_LAST_PHYS (0x3000000000) → TXM's own image pages fall outside its frame table

sptm_get_paddr_type() returns the out-of-range default → slab allocators can't carve from image memory

Fix: feed 64 real pages via AddFreeListPage before the first RegisterCodeSignature

# The whole binary rewriter: five patterns

four instructions that can't run at EL0, plus one hook to see TXM's own log

```
SVC #0      0xD4000001   TXM's SPTM hypercall (GENTER, encoded as SVC).
                         At EL0 it's a Darwin syscall.
                         → svc0_handler() decodes x16, fakes SPTM.

SVC #0x25   0xD40004A1   GL0 disable-interrupts trap.
                         → svc25_handler(), just logged.

SVC #0x26   0xD40004C1   GL0 enable-interrupts trap.
                         → svc26_handler(), just logged.

AT-GL0      0x00201440   Apple GXF address-translate (single site,
                         wrapped in ISB + TBNZ result,#0 fault-bit test).
                         Undefined at EL0.  → NOP  ("PA == VA").

txm_log     CMN W0,#1;   Not privileged, but TXM logs to a ring
            B.EQ;ADD…    buffer we can't see. Hooked so x22 (the
                         formatted "TXM [Log]: …") goes to stderr.
```

© 2026 calif.io · Off-by-One · Singapore

# Faking SPTM in ~200 lines

every rewritten SVC #0 lands here: decode x16, do one of six things

```
switch (x16 >> 56) {              // 27.0+ (DISPATCH_CMD_SHIFT=56)
case 0:   // DISPATCH_CALL  (table<<32 | endpoint)
  switch (x16) {
  case 0x100000001:  bootstrap done       → longjmp(txm_return)
  case 0x100000002:  register callback    → save dispatcher addr
  case 0x100000003:  retype(paddr, o, n)  → frame_type[paddr] = n
  } break;
case 1:   return-to-caller (26.x: 0xFD…)  → longjmp(txm_return)
case 2:   panic            (26.x: 0xFE…)  → print + crash
case 3:   exception state saved (27 W^X)  → longjmp
}
```

### Frame-type table

hash: paddr → SPTM frame type

TXM asserts types via SVC #0 too

originally hardcoded to two builds

agent added the version table + autodetect (learn shift from the first retype)

frame-type numbers change per release (26.x=+2, 27.0=+3), and 27.0 also changed the SPTM x16 ABI (command moved to x16[63:56]).

# Faking XNU: the bootstrap-args struct

~0x1B8 bytes: fill exactly what TXM reads, leave the rest zero, jump to entry

## Fields TXM actually reads

`device_tree_pa / _len`
real ADT, patched to point at the trust cache

`libsptm.first_phys / last_phys / n_papt_ranges`
bounds of the fake frame table

`libsptm.n_io_ranges = 0`
garbage → walks a garbage io_frame_table → SIGSEGV in Validate

`cpu_count = 1` · `thread_stacks[]`
cpu_count==0 → allocators never init → panic 0x07

`bootstrap_stack` · `max_asids`

**Calling into it**

```
// __txm_call(sel, a0..a6): assembly.s
MOV   W16, W0           // selector
MOVK  X16, #2, LSL #48  // TXM domain magic
LDR   X0, =txm_stack_base  // 16K, guard below
LDR   X8, =dispatcher_addr // from callback
BR    X8          // returns via SVC#0→longjmp
// result: sharedData.txmReturnWords[]
```

## The boot flow

bootstrap_txm(&args, entry) → TXM init → SVC#0 return → svc0_handler → longjmp → back in main()
TXM is now booted and ready to service selectors

# Handing it to a Coding Agent

the 2025 loader was hand-built for two specific TXMs. Then an agent got it.

### 2025 · hand-built

- 26 KB of C, written by hand
- Hardcoded to two TXM builds
- Frame-type table hardcoded
- Frame-type-assertion results hardcoded
- Break on the next point release

### 2026 · agent-expanded

- Dynamic frame-type table + version autodetect
- Boots every macOS 15.0 → 27.0 RC and iOS 26.x → 27.0 RC
- Fixed the 27.0 breakage: software W^X, new SPTM x16[63:56] ABI, shift +3
- Wrote PORTING.md so the next version is a 5-minute job

© 2026 calif.io · Off-by-One · Singapore

# It boots: macOS 27.0 RC (217.0.2)

```
TXM [Log]: restricted execution mode support: 0
TXM [Log]: successfully queried the UniqueDeviceIDv2 from the device tree
TXM [Log]: developer mode enabled due to system policy configuration
[SPTM] SPTM Register Callback (0, 0x100170418a4, 2)
[SPTM] SPTM Register Callback (1, 0x10017041710, 1)
TXM [Log]: build variant: txm.macosx.release.TrustedExecutionMonitor_Guarded-217.0.2
TXM [Log]: fusing: 0x00000000
TXM [Log]: virtualized: 0
TXM [Log]: build type: 0
TXM [Log]: device type: 0
TXM [Log]: research configuration: 0
TXM [Log]: extended research configuration: 0
[SPTM] TXM has finished and returned to caller
TXM FINISHED THE BOOTSTRAP NOW WE CAN CALL THE CALLBACK HANDLERS
[SPTM] SVC 0x25 - Disable Interrupts
[SPTM] SPTM Retype (0x110d74000, TXM_DEFAULT, TXM_FREE_LIST)
[SPTM] SVC 0x26 - Enable Interrupts
[SPTM] TXM has finished and returned to caller
[SPTM] SVC 0x25 - Disable Interrupts
[SPTM] SPTM Retype (0x110d78000, TXM_DEFAULT, TXM_FREE_LIST)
[SPTM] SVC 0x26 - Enable Interrupts
[SPTM] TXM has finished and returned to caller
        ... 244 more AddFreeListPage calls ...
[SPTM] SVC 0x25 - Disable Interrupts
[SPTM] SPTM Retype (0x110e70000, TXM_DEFAULT, TXM_FREE_LIST)
[SPTM] SVC 0x26 - Enable Interrupts
[SPTM] TXM has finished and returned to caller
[+] added 64 free-list pages to TXM
[+] thread-page snapshot taken (persistent-mode reset armed)
[SPTM] SVC 0x25 - Disable Interrupts
[SPTM] SVC 0x26 - Enable Interrupts
[SPTM] TXM has finished and returned to caller
txmNumReturnWords: 1
txmReturnWords[0]: 0x100170058e2  -> "txm.macosx.release.TrustedExecutionMonitor_Guarded-217.0.2"
[+] TXM boot complete (bootstrap + GetBuildVariant OK)
```

3

# The GLx Runner

© 2026 calif.io · Off-by-One · Singapore

# What is the GLx Runner

the real firmware chain, in a Hypervisor.framework guest, on your Mac

A macOS Hypervisor.framework guest that boots the REAL Apple guarded-world chain:

```
SPTM → TXM → Secure Kernel (cL4) → exclaves
```

Not an emulator. The actual signed SPTM/TXM Mach-Os from the IPSW execute in the guest.

On top: a gdbstub (attach lldb), block coverage, and a plugin API to drive it after boot.

the userland rehost is fast. This is faithful.

© 2026 calif.io · Off-by-One · Singapore

# How the GLx Runner happened

Hypervisor.framework, almost no public knowledge, and a lot of stubbornness

## Bootstrapped by hand

Started with ChatGPT in the browser + Apple's Hypervisor.framework

Nearly every public HVF demo was broken, incomplete, or just wrong: fixed manually until a vCPU actually ran guest code

Then one monitor at a time: SPTM → TXM → Secure Kernel (cL4) → exclave userland (roottask)

Each layer's base support = manual RE + trial-and-error against real firmware

## ...then handed to a coding agent

Port across versions (26.2 → 26.5.1 → 27.0 · macOS + iOS): 5 targets in core/target.c

Plugin system (plugin_api.h ABI v7, plugin_host.c): 26 C plugins, 4 400 lines

Python plugin bridge (plugin_python.c, 601 lines): 100% parity with the C ABI

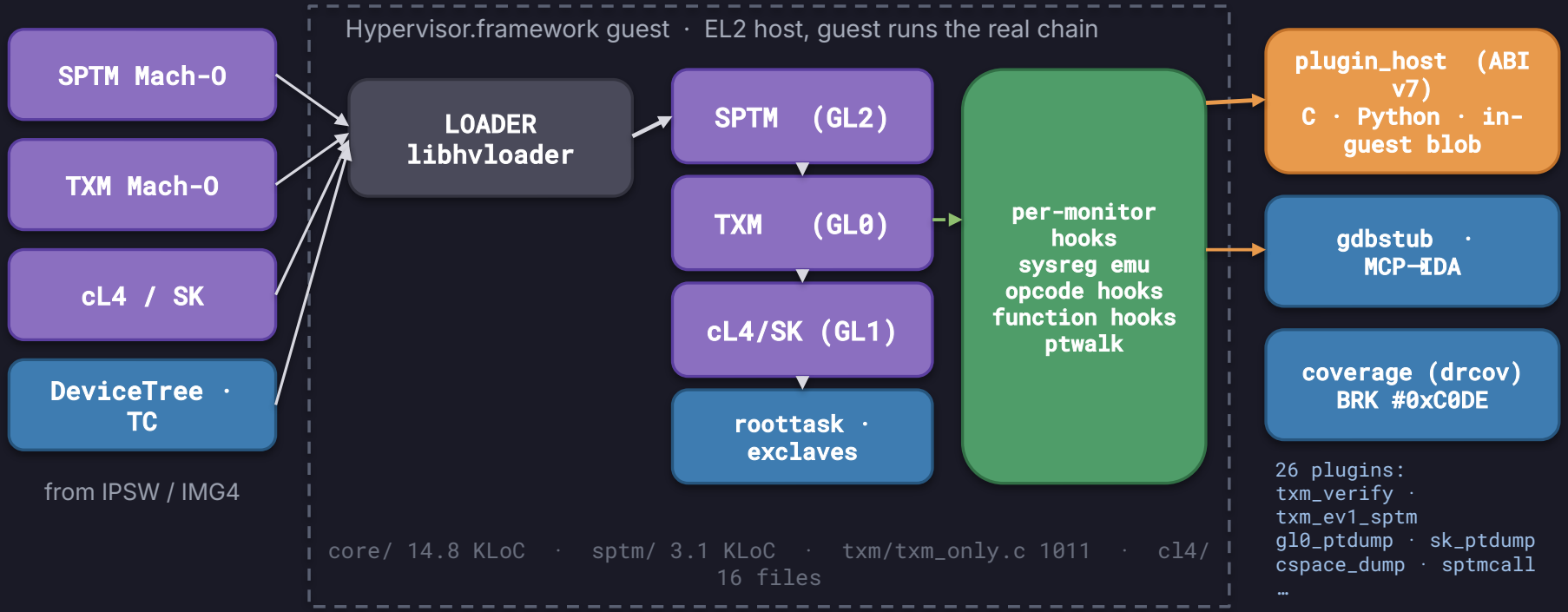
drcov coverage via self-healing BRK #0xC0DE at every block leader

gdbstub (2 741 lines), MCP → live IDA, ESR decoder, ptwalk

© 2026 calif.io · Off-by-One · Singapore

# GLx Runner architecture

the real firmware chain in a Hypervisor.framework guest: plugins drive it, gdbstub watches it



# Two axes, four modes

how GXF is executed × how much of the chain boots

### runner : emulated GXF

SPRR / GXF / GENTER trapped + faked (sysreg table, opcode hooks, PTE rewrite)

com.apple.security.hypervisor · any Apple Silicon Mac

### runner-vmapple : native GXF

hv_vm_config_set_isa(cfg, 3): private vmapple ISA → guest GXF runs on REAL silicon

SIP off + amfi_get_out_of_my_way + hvunlock.kext (clears bits 58/59)

### --txm-only : SPTM stubbed

TXM alone; SPTM SVCs stubbed (like the userland rehost, but in a guest with real page tables)

Fast iteration · this is what txm_verify.c drove for the audit's 11 tests

### full boot : SPTM boots TXM

Real SPTM builds the thread stacks + guard pages + frame table

The plugin plays XNU after SPTM's final GEXIT · this is txm_ev1_sptm.c

© 2026 calif.io · Off-by-One · Singapore

# Real vs. faked

the userland rehost is fast; the GLx Runner is TRUE

## userland rehost

- TXM Mach-O in a macOS process
- Every SPTM call = a C stub (retype, va↔pa, panic)
- No page tables, no GL levels, no real guard pages
- Thread stack = whatever mmap gave you
- SPTM panic = *(char*)0x333 = 0x44
- `~1 340 exec/s : built for fuzzing`

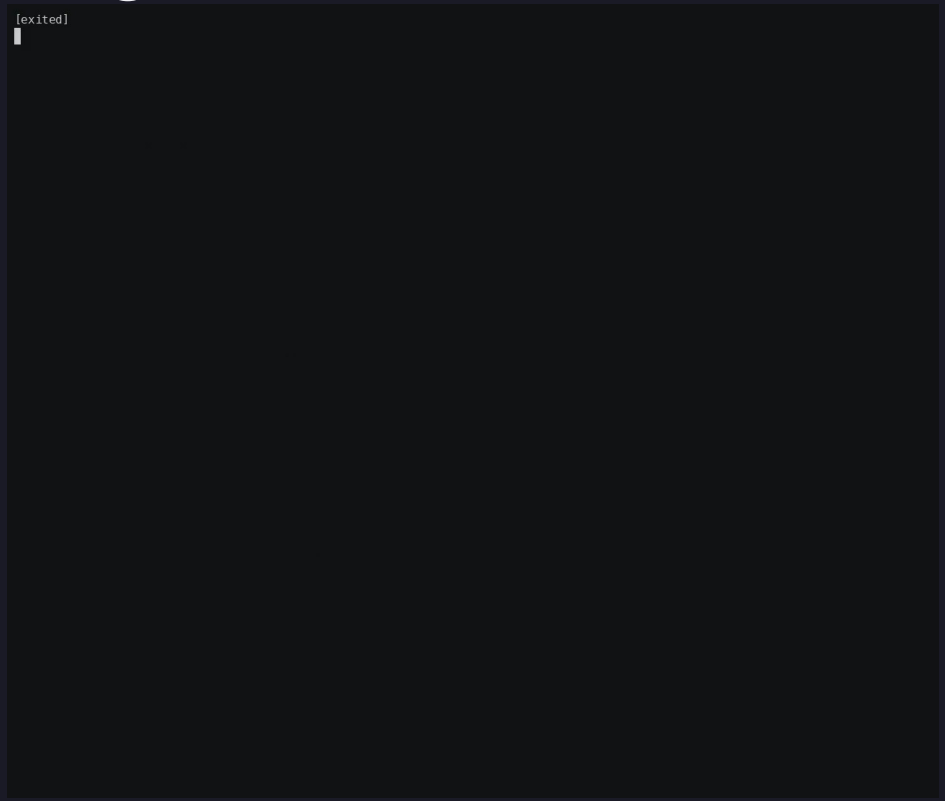
## GLx Runner

- Real SPTM binary enforces retype / frame-type ownership
- Real GENTER / GEXIT dispatch (emulated or on-silicon)
- Real SPTM-built thread stacks: L3 PTE=0 guard pages, ptwalk-verifiable
- Real page tables · real SPTM panic handler
- A crash here IS a crash on hardware
- `~seconds/run : built for proof`

the rehost "survived only because the harness maps the page below the stack": real SPTM doesn't.

© 2026 calif.io · Off-by-One · Singapore

# Instrumenting it: lldb on real TXM at GL0



4

# Diffing TXM Across Releases

# No Public CVE in TXM

as far as we can tell: not one, in three years of shipping



public CVEs · public bug writeups

## What that means

Shipping since 2023

PPL got years of public research

TXM got none

Nobody has looked, or nobody has talked

# So we diffed every release

no public info, but the code kept changing.  Silent fixes leave a trail.

## The idea

Nothing published, but three years of point releases

If Apple fixed something quietly, the fix is in the binary diff

Pull every TXM, diff each adjacent pair, read the changed functions

## What we pulled

macOS  15.0 → 26.6.2 → 27.0 RC

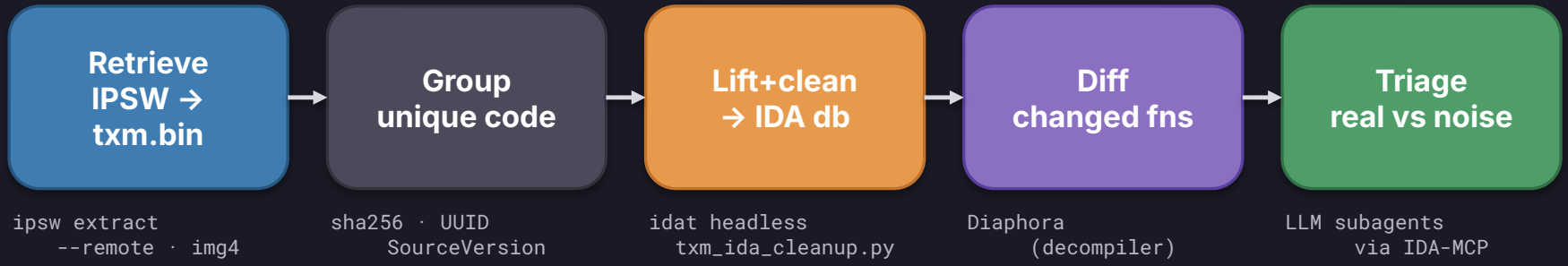
iOS  18.6.2 → 26.6.1 → 27.0 RC

28 IPSWs → 13 unique-code TXM binaries

TXM builds:  135.x · 182.x · 187.x · 215 · 217.0.2

# The Diffing Pipeline

the exact toolchain the agents drove: from IPSW to a triaged finding




Version discovery via appledb · remote extraction → no full-IPSW downloads · TXM-specific IDA cleanup (SVC-panic noreturns, function-boundary splitting) · the whole pipeline scripted and driven by parallel AI subagents.

© 2026 calif.io · Off-by-One · Singapore

# How good is the diff signal?

before trusting a changed-function list, we checked what "changed" actually means

IDA cleanup script fixes TXM's SVC noreturns → correct function boundaries
Code coverage ~98%; forcing it to 99.7% moved the counts by <10%
diaphora over decompiled pseudocode, identical functions filtered out

“changed” = reliable signal, worth reading
“unmatched” = a diaphora matching heuristic, NOT removed code

# Debug-Region Exec Bypass

HIGH · 15.3 → 15.4 · fixed: an unsigned-code-execution primitive

## What it is

TXM's SPTM callback decides whether a page may get W/X, per SPTM frame type

For a debug-region page (frame-type 15), 15.3 checks one bit first: the address space's “debug-allowed” flag

If that bit is set, the entire permission check is skipped

→ PROT_EXEC granted with no developer-mode and no code-region check: unsigned code past TXM, in the malicious-kernel model

## The 15.4 fix

EXEC now requires the page to sit in a registered code-region (flag 0x80)

WRITE always requires developer_mode == 1

The debug bit is demoted to a write-only entitlement substitute

Verified from decompilation. Real policy-gap fix; whether a malicious kernel can set the debug bit is not proven here.

© 2026 calif.io · Off-by-One · Singapore

# SetSharedRegionBaseAddress TOCTOU

candidate · 15.6 → 26.0 · NOT backported: still shipping in 15.4–15.6.1

## What it is

The setter writes an unsynchronised global {base, base+size}: no lock, no set-once guard

The consumer bounds-checks that global…

…then, after intervening work, RE-READS the same global into a code-region object that governs executable mappings

A 2nd CPU re-issuing the selector in that window could stamp an out-of-window range into an exec mapping

## The 26.0 fix

Per-address-space field, not a global

Locked + set-once (panic 135 on re-set)

Read once into locals; new bounds panic 137

Code shape verified; still unpatched in 15.x. The precondition (two CPUs in TXM at once) isn't provable from the binary.

© 2026 calif.io · Off-by-One · Singapore

# Trust-Cache Lock → Panic

DoS · 15.0 → 15.1 · fixed

## What it is

The trust-cache list used a home-grown non-blocking RW-lock byte

On contention it calls txmPanicWithCode(54) instead of waiting

QueryTrustCache runs inside every code-signature validation

A malicious kernel races LoadTrustCache against a QueryTrustCache → TXM panic → device halt

## The 15.1 fix

Replaced with a lock-free singly-linked list

release STLR/CASL insert, acquire LDAPR walk

Contention resolves by retry, not panic

Verified in both directions. DoS only, no memory corruption; both actors are kernel-reachable selectors.

© 2026 calif.io · Off-by-One · Singapore

# CoreEntitlements Key Aliasing

hardening · added 26.3 → 26.4

## What it is

Entitlements are a DER-encoded dictionary; canonical DER requires strictly-sorted, unique keys

26.3's validator checked each entry in isolation: no key-to-key comparison

A blob could carry two entries for the same key, or non-canonical ordering

TXM and a downstream consumer (AMFI / sandbox) could then disagree on which value wins: a parser-differential

## What 26.4 added

After each entry, memcmp its key vs the previous key

Reject non-ascending (err 208) and exact duplicates (err 4)

Plus per-key UTF-8 / non-empty / no-NUL checks

Verified hardening (function names are ours; binary is stripped). Concrete exploitability needs cross-component analysis: unproven.

© 2026 calif.io · Off-by-One · Singapore

# Further Findings

lower-tier changes the diffs also surfaced: each re-verified from decompilation

## Also found (and confirmed)

LoadTrustCache copy-before-validate: payload copied into TXM-owned memory before validation, so the kernel can't swap it post-check (hardening · 15.4)

SRD / ERM entitlement gates: 26.0 blocks research entitlements + ERM trust caches on non-research devices; NOT backported to 15.x

Inverted apfs-preboot-uuid check: 26.0's hex validator was flipped → panic; boot-DeviceTree only, not kernel-reachable (fixed 26.1)

ccec_der_import_pub 1-byte OOB read: empty BIT STRING; stays inside a larger DER buffer, no fault / leak (defensive · 15.2)

© 2026 calif.io · Off-by-One · Singapore

5

# Auditing TXM with Agents

© 2026 calif.io · Off-by-One · Singapore

# 20 parallel auditors, one IDB each

diffing shows what Apple already fixed.  The audit reads the CURRENT binary and asks for bugs.

## The prompt (AUDIT_BRIEF.md)

"You are one of ~20 parallel auditors" : 18 deep + 1 surface-mapper + 2 cross-cutting sweeps

Each gets its OWN private IDA .i64, driven live via idasql (SELECT decompile(0x...))

Partitioned by SUBSYSTEM, not by function: TrustCache / Image4-DER / CodeSig / Entitlements / ...

Explicit threat model + bug-class taxonomy + 11 historical-fix seeds ("does the same pattern exist in MY parser?")

Anti-hallucination block: "quote real code: every claim cites a /* addr */ line from decompile()" + a list of Firebloom / PAC / wide-pointer patterns to ignore

## The output contract

1 parsers/<tag>_analysis.md evidence file per auditor

1 findings/<tag>_<N>_<slug>.md per finding: fixed 9-section template

Every finding independently RE-VERIFIED by two skeptic passes (2 × 61)

Target: 187.120.2 · 1325 fns · 172 named · Firebloom-compiled

31 IDBs on disk (20 main + 4 verify + 7 iOS-27 follow-on)

© 2026 calif.io · Off-by-One · Singapore

# 61 findings → 53 survived → 1 high

"No trust bypass and no attacker-controlled memory-write primitive was found."

**2-skeptic filtered**

| | surv | refuted |
|---|---|---|
| high | 1 | 1 |
| medium | 1 | 1 |
| low | 21 | 4 |
| info | 30 | 2 |
| | 53 | 8 |

mem-corr 8 (1 live)
logic 34 · leak 11

## The one HIGH

CoreEntitlements V2 validator crash

Still unfixed in 27.0 RC

Apple: not a security issue, it just crashes

Not triggerable from userspace right now: another component panics on the same input before TXM ever gets it

## What the model got wrong

Misunderstood SPTM frame-type access rights (assumed a caller could use types it can't)

Frame-type numbering (changed between versions; auditors used stale numbers)

8 findings refuted by the skeptic pass, each with a concrete counter-proof

© 2026 calif.io · Off-by-One · Singapore

# …then we ran them

every high/medium/low finding replayed against live TXM via GLXRunner's txm_verify plugin

## 11 verification tests → 5 verdict classes

ENT_VALIDATE-1 CONFIRMED: 128 recursions, SP crossed the page boundary at frame #113, exactly 0x80/frame

ENT_VALIDATE-2, DISPATCH-5, PROV-2, INFOLEAK-1 CONFIRMED

DISPATCH-4a REFUTED · IMG4_DISPATCH-3 COULD-NOT-TEST

ptwalk on booted real SPTM: L3 PTE=0 below all 10 TXM thread stacks → guard page is real → DoS-only

XNU can read TXM memory directly → downgraded the whole info-leak class to hygiene

## What ONLY running it caught

The static audit's call-chain was wrong: sel 22 → validator? No: breakpoint at CEValidateWithOptions fires on sel 24, never sel 22.

Precondition the static read missed: CD needs nSpecialSlots ≥ 7 and slot −7 == SHA256(entitlements), else the validator is silently skipped.

The rehost/--txm-only harness "survived only because the harness maps the page below the stack": real SPTM doesn't.

the audit's own conclusion: the parsers are Firebloom-hardened, static reading finds little more. Fuzz them.

© 2026 calif.io · Off-by-One · Singapore

6

# From Rehost to Fuzzers

# 26 KB → 74 KB: what the agent added

the 2025 hand-built loader, handed to a coding agent with one prompt: "turn this into a fuzzer"

## The diff

```
txm_loader.c    697 → 1847 lines
functions        22 → 40
getopt    -t -d -c -h
        → 14 flags
assembly.s      137 → 218
+  afl_cov.c/.h    341 lines
+  6 markdown docs
+  6 python tools
```

## What it wrote (all agent, no human edits)

- 5 per-input harnesses: harness_run_{input,image4,profile,trustcache} + run_bootstrap_replay
- AFL++ new-style forkserver (FDs 198/199, FS_NEW_OPT_MAPSIZE) + shm map
- Two coverage backends: frida-mode wrapper + a native basic-block trampoline instrumenter
- collect_corpus.py: fingerprints CS blobs by structure, keeps ≤N per shape
- run_all_fuzzers.sh: 7 surfaces in parallel, one tmux pane each
- drcov emitter → IDA Lighthouse (used to find the img4/profile parser gates)
- plant_bug.py: patches sanitizer_runtime_violation → UDF #0 to validate the pipeline

the human wrote the 2025 rehost. Everything on this slide is the agent's.

© 2026 calif.io · Off-by-One · Singapore

# Which selectors take attacker bytes

52 selectors → 4 hand TXM attacker-controlled bytes via txm_transfer_region() → 5 harnesses

```
-b  RegisterCodeSignature       CS SuperBlob     CD · Reqs · CMS · XML/DER
    + ValidateCodeSignature                       entitlements · launch constraints
-4  Image4Dispatch/ACTIVATE     IM4M manifest    ASN.1 DER + X.509 cert chain
                                                  "the richest untouched surface"
-p  RegisterProvisioningProfile .provisionprofile CMS-signed DER plist
-T  LoadTrustCache              trust-cache blob  runtime ABI: type/length/addr
-B  (bootstrap DT trust cache)  trust-cache       ONLY path to the module parser
                                container         = the DFU/recovery boot path
```

## Gates it found (via drcov) and what it did about them

img4  bails at error 0x54 pre-decode → wrote img4_open_gate.py (patch one CBZ) → corpus 5→63 in 40 s, ~1530 vs ~1110 blocks. Left the 0x50 CoreTrust gate closed on purpose (all ASN.1 already runs before it).

profile  0xF1 in CTVerifyAmfiCMS: nested crypto, NOT patchable.  Fuzz with TXM_IGNORE_PANIC=1 → 13→181 in 30 s.

trust cache  deep-load ends at a SecureChannel op.  Real parser has ONE caller: the bootstrap DT loop → hence -B.

© 2026 calif.io · Off-by-One · Singapore

# Building the corpus

one structurally-diverse corpus per surface: real blobs, not random bytes

## collect_corpus.py (CS SuperBlobs)

Harvest local FS + mounted IPSW rootfs + a codesign generation matrix (19 shapes)

Fingerprint each by (slots, CD ver, hash type, CS flags, nSpecial): keep ≤3 per bucket

→ ~57 distinct structural shapes on this Mac alone

**What that produced**

| | | |
|---|---|---|
| seeds/ | 47 | real LC_CODE_SIGNATUREs |
| seeds_corpus/ | 143 | collect_corpus.py output |
| seeds_img4/ | 6 | apticket.*.im4m (ipsw) |
| seeds_profile/ | 13 | embedded.provisionprofile |
| seeds_trustcache/ | 3 | full.tc + split modules |

## The bit that mattered later

None of these seeds carry a nested-DER entitlements slot at any depth: the CE-recursion path was structurally unrepresented in the corpus.

That's not a corpus bug; it's why blind mutation never got there.

© 2026 calif.io · Off-by-One · Singapore

# Hooking it up to AFL

two backends, three dead ends, one 100× speedup

## frida-mode (afl-fuzz -O)

~15 exec/s

Real edge coverage via frida-gum Stalker

Cost = re-stalking TXM's bootstrap every fork

Tried deferred fork → ~1 exec/s (17 s cold recompile per child)

Tried persistent → ~2 exec/s (2.7% of inputs panic → batch dies at ~37)

## native forkserver (NATIVE=1)

~1 340-1 500 exec/s (100×)

Loader owns the forkserver (afl_cov.c): no Stalker at all

Phase 2b: template trampolines at every block leader

12 078 leaders hooked from an IDA-exported block table

clobber only x0-x5+NZCV; PC-relative leaders skipped (safe by construction)

256 K map (12 K blocks would saturate 64 K)

## Two gotchas the agent documented

Never set AFL_SKIP_BIN_CHECK on the native backend: forces afl-fuzz into dumb mode (bitmap 100%, zero finds). The forkserver reporting a map size IS the instrumentation signal.

AFL_MAP_SIZE in afl_cov.h and run_afl.sh must match: 12 K blocks × edge hashing needs 256 K.

© 2026 calif.io · Off-by-One · Singapore

# Running it: what actually fell out

7 surfaces in parallel, ~1300 exec/s each · many crashes, all controlled panics · 0 real ones

## Per surface

- cs-blob: ~16 unique 0xF1 kTXMPanicCodeSignature in 20 min: Firebloom bounds-checks catching mutations
- -B replay: 14 crashes in 60 s (10× 0xF0, 3× 0x23-0x25): all controlled TXM panics
- TXM_IGNORE_PANIC=1 across all 7 → 0 saved crashes
- planted UDF #0 at sanitizer_runtime_violation: NOT reached: the corpus plateaus before the deep CD checks

## Why the ONE known bug wasn't among them

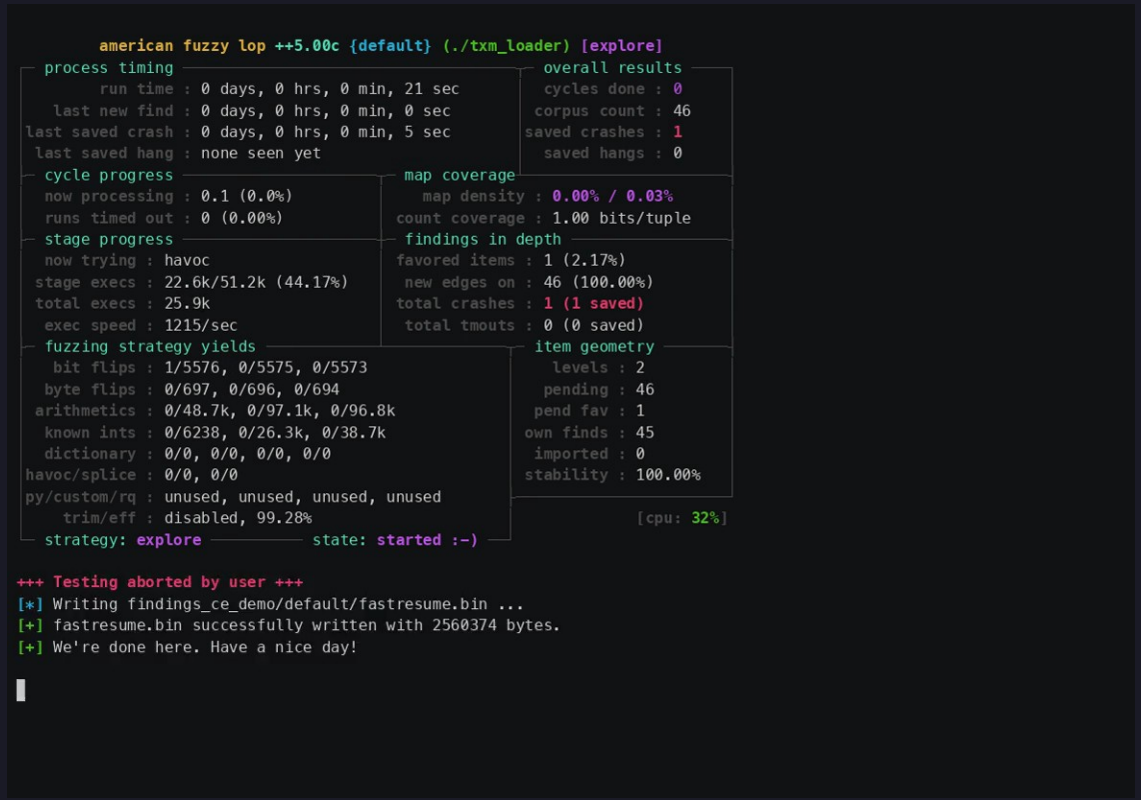
- 16 KiB stack overflow: but the loader gave TXM ~1 MB of stack
- Gated by CD slot-7 SHA-256: any DER mutation → validator silently skipped
- Outcome-edge coverage said every DER byte looked identical to AFL

## Three knobs that fixed it

| | |
|---|---|
| `TXM_STACK_GUARD=1` | `mmap [guard][16 KiB]: the real SPTM layout. Threshold = depth 113, matches hardware.` |
| `TXM_FIXUP_ENT_HASH=1` | `recompute every CD special-slot hash after mutation → DER changes pass the gate.` |
| `+2 afl_cov_edge()` | `feed CSReturn component/error into the map → skipdet marks DER bytes effective.` |

© 2026 calif.io · Off-by-One · Singapore

# AFL finds it: CoreEntitlements V2 recursion



# Replaying the AFL crash

```
TXM [Log]: successfully queried the UniqueDeviceIDv2 from the device tree
TXM [Log]: developer mode enabled due to system policy configuration
TXM [Log]: build variant: txm.macosx.release.TrustedExecutionMonitor_Guarded-217.0.2
TXM [Log]: fusing: 0x00000000
TXM [Log]: virtualized: 0
TXM [Log]: build type: 0
TXM [Log]: device type: 0
TXM [Log]: research configuration: 0
TXM [Log]: extended research configuration: 0
TXM FINISHED THE BOOTSTRAP NOW WE CAN CALL THE CALLBACK HANDLERS
[+] TXM thread stack: 16 KiB @ 0x110810000, guard @ 0x11080c000
[+] added 64 free-list pages to TXM
[+] thread-page snapshot taken (persistent-mode reset armed)
txmNumReturnWords: 1
txmReturnWords[0]: 0x100170058e2  -> "txm.macosx.release.TrustedExecutionMonitor_Guarded-217.0.2"
[+] loaded raw CS blob: 697 bytes from findings_ce_demo/default/crashes/id:000000,sig:04,src:000000,time:16405,execs:197
32,op:havoc,rep:1
[+] GetCodeSigningInfo: managed signature size = 0x2000
[+] RegisterCodeSignature -> rc=0x0 sig_obj=0x114fc7dd0 txm_sig_addr=0x114fc3c00

── TXM CRASH — signal 10 (SIGBUS) ──────────────────────────────────────
  fault addr : 0x00011080ff80  (guard page: stack_base - 0x80)
  pc         : 0x01001707f7f8  (TXM 0xfffffff01707f7f8)
  lr         : 0x01001707fbd0  (TXM 0xfffffff01707fbd0)
  sp         : 0x00011080ff40  (192 bytes into guard)
  fp         : 0x000110810080
  backtrace  :
     #0   0x01001707fbd0  (TXM 0xfffffff01707fbd0)
     #1   0x01001707fd34  (TXM 0xfffffff01707fd34)
     ... 110 more frames at 0x01001707fd34  (TXM 0xfffffff01707fd34) ...
     #112 0x01001707f96c  (TXM 0xfffffff01707f96c)
     #113 0x01001707f440  (TXM 0xfffffff01707f440)
     #114 0x010017046d44  (TXM 0xfffffff017046d44)
     #115 0x01001704a914  (TXM 0xfffffff01704a914)
     #116 0x01001704b988  (TXM 0xfffffff01704b988)
     #117 0x0100170388e8  (TXM 0xfffffff0170388e8)
     #118 0x01001703f22c  (TXM 0xfffffff01703f22c)
     #119 0x0001044a3ba0
─────────────────────────────────────────────────────────────────────────
```

# Proving it: the CE crash in the hypervisor

plugins/txm_ev1_sptm · 309 lines · the plugin plays XNU, SPTM's own panic handler fires

```
on_boot_complete:           // SPTM's final GEXIT: plugin plays XNU
 8× sel 10 AddFreeListPage       // type_frame(pa,1,TXM_TRANSFER)
 1× sel 22 RegisterCodeSignature // SuperBlob, slot-7 nested DER
                                 //   staged into idle tstack[1]
 1× sel 24 ValidateCodeSignature // → CEContextInitWithType
                                 //   → ce_validate_element
                                 //   → SPTM's GL0 guard page

X16 = x0002_0000_0000_0000 | selector   // RE'd from XNU txm_enter
X0  = thread_stack PA   →   svc->sptm_genter(vcpu, regs)
```

## The result

ELR 0xFFFFFFF017048744

FAR 0xfeafff8

= stack_base − 8

ESR 0x96000047

SPTM panic:

"[TXM] Unhandled synchronous exception taken from GL0"

# SPTM's own panic: live on 27.0 RC

```
TXM LOG: setup logging: 32768 bytes (256 | 128)
TXM LOG: system supports DIT feature
TXM LOG: unable to find esdm-fuses property in /chosen
TXM LOG: unable to find vmm-present property in /defaults
TXM LOG: resolved system platform identity: 0
TXM LOG: Code Signing Monitor Image4 Module Version 7.0.0: Sat Aug  8 12:39:48 PDT 2026; root:AppleImage4_txm-374~7020/libimage4_TXM
/RELEASE_ARM64E
TXM LOG: libimage4: no boot uuid set
TXM LOG: unable to find sepfw-never-boot property in /chosen
TXM LOG: unable to find research-enabled property in /chosen
TXM LOG: unable to find research ERM property in /chosen
TXM LOG: unable to find iboot-build-variant property in /chosen/iBoot
TXM LOG: loaded external trust cache modules: 1/1
TXM LOG: unable to find amfi-only-platform-code property in /chosen
TXM LOG: platform code only policy: 0
TXM LOG: restricted execution mode support: 0
TXM LOG: successfully queried the UniqueDeviceIDv2 from the device tree
TXM LOG: developer mode enabled due to system policy configuration
TXM LOG: build variant: txm.macosx.release.TrustedExecutionMonitor_Guarded-217.0.2
TXM LOG: fusing: 0x00000000
TXM LOG: virtualized: 0
TXM LOG: build type: 0
TXM LOG: device type: 0
TXM LOG: research configuration: 0
TXM LOG: extended research configuration: 0

═══════ SPTM boot complete — harness now plays XNU ═══════
  [ev1] SPTM built 10 TXM thread stacks · running on tstack[0] VA=0xfffff7f000284000  guard@-0x4000 = PTE 0
  [ev1] GENTER sel 10 AddFreeListPage ×8  (feed TXM slab allocator)  → rc=0
  [ev1] staged malformed SuperBlob (761 B, slot-7 DER = 128 nested SEQUENCEs) @ TXM VA 0xfffff7f00028c000
  [ev1] GENTER sel 22 RegisterCodeSignature(blob=0xfffff7f00028c000, 761)
  [ev1]        ← rc=0  handle=0xfffff7f00001fdd0  sig[0..8]=fade0cc0 000002f9
  [ev1] GENTER sel 24 ValidateCodeSignature(0xfffff7f00001fdd0)  → CEValidate → ce_validate_element …

panic("panic_with_register_state: [DISPATCH_ID_SPTM_CORE] [TXM] Unhandled synchronous exception taken from GL0 at pc 0xfffff8002fca9
b4c, lr 0xfffff8002fccbd24
         x0:  0xfffff7f000284010 x1:  0xfffff7f000284020  x2:  0x0000000000000000  x3:  0x0000000000000001
         x4:  0x0000000000000020 x5:  0x0000000000000000  x6:  0x0030023004300630  x7:  0x0000000000000000
         x8:  0xfffff7f00001bedb x9:  0xfffff7f00001bef9  x10: 0xfffff7f00001bef9  x11: 0x000000000000001e
         x12: 0xfffff7f00001bedb x13: 0x000000000000001e  x14: 0xfffff7f00001beda  x15: 0x000000000000001e
         x16: 0x0000000000000080 x17: 0x0000000000000000  x18: 0x00000000000001c0  x19: 0xfffff7f000287a68
         x20: 0x0000000000000000 x21: 0x0000000000000000  x22: 0xdfffffffffffffe0  x23: 0xa000000000000010
         x24: 0x0000000000000000 x25: 0x0000000000000000  x26: 0x2000000000000010  x27: 0x0000000000000000
         x28: 0x0000000000000008 fp:  0xfffff7f000284080  lr:  0xfffff8002fccbd24  sp:  0xfffff7f000283fd0
         pc:  0xfffff8002fca9b4c cpsr: 0x800003c0         esr: 0x0000000092000047         far: 0xfffff7f000283ff0
         LLC_ERR_STS: 0")
```



# If you have any questions, please contact

stefan@calif.io · @i0n1c

© 2026 calif.io · Off-by-One · Singapore